

### Звіт з лабораторної роботи № 3

Поліморфізм. Створення та використання ієрархії класів. Віртуальні функції  
11 варіант

1. Мета: Розробити ієрархію класів згідно отриманого завдання описати їх алгоритмічною мовою C#. Написати фрагмент програми з використанням об'єктів створених класових типів.
2. Завдання: автомобіль, потяг, транспортний засіб, експрес
- 2.1. Прописуємо клас AutomatedTransport. Ось його код:

```
internal class AutomatedTransport
{
    protected string Manufacture;
    protected string Model;
    protected int Price;

    public AutomatedTransport()
    {
        Manufacture = "None";
        Model = "None";
        Price = 0;
    }

    public AutomatedTransport(string man, string mod, int pr)
    {
        Manufacture = man;
        Model = mod;
        Price = pr;
    }

    public virtual void AddInfo()
    {
        Console.WriteLine("| Enter manufacture: ");
        Manufacture = Console.ReadLine();
        Console.WriteLine("| Enter model: ");
        Model = Console.ReadLine();
        Console.WriteLine("| Enter price:");
        Price = Int32.Parse(Console.ReadLine());
    }

    public virtual void OutInfo()
    {
        Console.WriteLine("| " + Manufacture + " | " + Model + " | " + Price + " |");
    }

    public virtual string return_(int s)
    {
        switch (s)
        {
            case 2: return Model;
            case 3: return Price.ToString();
            default: return Manufacture;
        }
    }
}
```

#### 2.2. Пояснення:

- 2.2.1. У класі є такі атрибути: Manufacture, Model, Price.
- 2.2.2. У конструкторах описані початкові значення атрибутів.
- 2.2.3. У класі є такі методи: AddInfo() – дозволяє додати інформацію про транспортний засіб, OutInfo() – дозволяє вивести інформацію про транспортний засіб, return\_() – повертає значення поля у відповідності з обраним s.

|           |                 |          |        |      |                                                                               |  |  |      |       |         |
|-----------|-----------------|----------|--------|------|-------------------------------------------------------------------------------|--|--|------|-------|---------|
|           |                 |          |        |      | ЛР3.14.КІ-202.11.03                                                           |  |  |      |       |         |
| Змін.     | Аркуш           | № докум. | Підпис | Дата |                                                                               |  |  |      |       |         |
| Розробив  | Куліков М. М..  |          |        |      | Поліморфізм. Створення та використання ієрархії класів.<br>Віртуальні функції |  |  | Літ  | Аркуш | Аркушів |
| Перевірив | Ковальчук С. О. |          |        |      |                                                                               |  |  | Н    | 1     | 5       |
| Н. Контр. |                 |          |        |      |                                                                               |  |  | ХПФК |       |         |
| Затвер.   |                 |          |        |      |                                                                               |  |  |      |       |         |

### 2.3. Прописуємо класи Auto, Train, Express. Ось їх код:

```
internal class Train : AutomatedTransport
{
    protected int Number;
    public Train()
    {
        Manufacture = "None";
        Model = "None";
        Price = 0;
        Number = 1;
    }
    public Train(string man, string mod, int pr, int num)
    {
        Manufacture = man;
        Model = mod;
        Price = pr;
        Number = num;
    }

    public override void AddInfo()
    {
        base.AddInfo();
        Console.WriteLine("| Enter number:");
        Number = Int32.Parse(Console.ReadLine());
    }

    public override void OutInfo()
    {
        Console.WriteLine("| " + Manufacture + "-" + Model + " | " + Price + " | N:" + Number + " |");
    }
}

internal class Auto : AutomatedTransport
{
    public String NumOnPlate;
    public Auto()
    {
        Manufacture = "None";
        Model = "None";
        Price = 0;
        NumOnPlate = "AT0001AT";
    }

    public Auto(string man, string mod, int pr, string numbpl)
    {
        Manufacture = man;
        Model = mod;
        Price = pr;
        NumOnPlate = numbpl;
    }

    public override void AddInfo()
    {
        base.AddInfo();
        Console.WriteLine("| Enter number:");
        NumOnPlate = Console.ReadLine();
    }

    public override void OutInfo()
    {
        Console.WriteLine("| " + Manufacture + " | " + Model + " | " + Price + " | " + NumOnPlate + " |");
    }
}

internal class Express : Train
{
    public Express()
    {
        Manufacture = "None";
        Model = "None";
        Price = 0;
        Number = 1;
    }

    public Express(string man, string mod, int pr, int num)
    {
        Manufacture = man;
        Model = mod;
        Price = pr;
        Number = num;
    }

    public override void AddInfo()
    {
        base.AddInfo();
        Console.WriteLine("| Enter number:");
        Number = Int32.Parse(Console.ReadLine());
    }
}
```

## 2.4. Пояснення:

- 2.4.1. У класах є такі атрибути: Manufacture, Model, Price (успадковані від батьківського). У Train ще є Number, я і в спадкового Express. У Auto ще є NumOnPlate.
- 2.4.2. У конструкторах описані початкові значення атрибутів.
- 2.4.3. У класі є такі методи:

- Успадковані/перезаписані: AddInfo() – дозволяє додати інформацію про транспортний засіб, OutInfo() – дозволяє вивести інформацію про транспортний засіб.

## 2.5. Прописуємо меню. Ось його код:

```
string end = "-----";
int k = 0, l = 0;
//Список Авто
List<AutomatedTransport> veh = new List<AutomatedTransport>();
veh.Add(new AutomatedTransport("Horse wagon", "4Wheeled", 2000));
veh.Add(new Auto("Ford", "Transit", 20000, "BX7777BX"));
veh.Add(new Auto("Mercedes", "G65 AMG", 2000000, "G65"));
veh.Add(new Train("VL", "10", 20000, 01));

while (k != 1)
{
    //Меню і основний код
    Console.WriteLine("-----| Menu |-----");
    Console.WriteLine("1.Add new vehicle");
    Console.WriteLine("2.Delete vehicle");
    Console.WriteLine("3.Show vehicle list");
    Console.WriteLine("4.Show vehicle list by type");
    Console.WriteLine("5.Show vehicle in vehicle list by index");
    Console.WriteLine("6.Sort");
    Console.WriteLine("7.Search");

    Console.WriteLine("0.Exit");
    Console.WriteLine(end);
    Console.WriteLine("Choose an option: ");

    l = Int32.Parse(Console.ReadLine());

    Console.WriteLine(end);
    switch (l)
    {
        //виведення по типах
        case 0:
        {
            k = 1;
            break;
        }
        case 1:
        {
            AutomatedTransport r = new AutomatedTransport();
            r.AddInfo();
            veh.Add(r);
            Console.WriteLine(end);
            break;
        }
        case 2:
        {
            int t = 0;
            Console.WriteLine("Enter a index of vehicle:");
            t = Int32.Parse(Console.ReadLine());
            t = t - 1;
            veh.RemoveAt(t);
            Console.WriteLine(end);
            break;
        }
        case 3:
        {
            foreach (AutomatedTransport p in veh)
            {
                p.OutInfo();
            }
            Console.WriteLine(end);
            break;
        }
        case 4:
        {
            Console.WriteLine("Enter what you choose: ");
            string word = Console.ReadLine();
            Console.WriteLine(end);
            foreach (AutomatedTransport p in veh)
            {
                if(word == getTyp(p))
                {
                    - - - -
                }
            }
        }
    }
}
```

|     |      |          |        |      |
|-----|------|----------|--------|------|
|     |      |          |        |      |
| Вим | Арк. | № докум. | Підпис | Дата |

```

        p.OutInfo();
    }
}
Console.WriteLine(end);
break;
}
case 5:
{
    Console.WriteLine("Enter a index of vehicle:");
    int t = (Int32.Parse(Console.ReadLine()) - 1);
    veh[t].OutInfo();
    Console.WriteLine(end);
    break;
}
case 6:
{
    int size = 0;
    foreach (AutomatedTransport p in veh)
    {
        size++;
    }
    S();
    Console.WriteLine("Choose an option: ");
    int s = Int32.Parse(Console.ReadLine());
    Sort(veh, size, s);
    Console.WriteLine(end);
    Console.WriteLine("Finished");
    Console.WriteLine(end);
    break;
}
case 7:
{
    AutomatedTransport r = new AutomatedTransport();
    Console.WriteLine(end);
    Console.WriteLine("Search: ");
    S();
    Console.WriteLine("Choose an option: ");
    int s = Int32.Parse(Console.ReadLine());
    Console.WriteLine("Enter what you choose: ");
    string word = Console.ReadLine();
    Console.WriteLine(end);
    foreach (AutomatedTransport p in veh)
    {
        Search(word, p, s);
    }
    Console.WriteLine(end);
    break;
}
default:
{
    Console.WriteLine("Try again");
    Console.WriteLine(end);
    break;
}
}
if (l != 0)
{
    Console.WriteLine("Press enter for return to menu: ");
    Console.ReadLine();
    Console.Clear();
}
}

```

## 2.6. Пояснення:

2.6.1. Меню працює доти, поки змінна k не буде дорівнювати 1. Коли k=1, то меню зупиняється.

## 2.7. Прописуємо код сортування та пошуку. Ось його код:

```

public static void Sort(List<AutomatedTransport> list, int size, int s)
{
    for (int i = 0; i < size; i++)
    {
        int max_i = i;
        string max = list[i].return_(s);
        for (int j = i + 1; j < size; j++)
        {
            string sec = list[j].return_(s);
            if ((s == 1) || (s == 2))
            {
                if (String.Compare(max, sec) > 0)
                {
                    max_i = j;
                }
            }
        }
    }
}

```

```

    }

    else if ((s == 3))
    {
        if (int.Parse(max) > int.Parse(sec))
        {
            max_i = j;
        }
    }
    max = list[max_i].return_(s);
}

AutomatedTransport list_sort = list[max_i];
list[max_i] = list[i];
list[i] = list_sort;
}

}

public static void S()
{
    Console.WriteLine("1.By manufacture");
    Console.WriteLine("2.By model");
    Console.WriteLine("3.By price");
}

public static void Search(string word, AutomatedTransport p, int s)
{
    if (word.Equals(p.return_(s)))
    {
        p.OutInfo();
    }
}

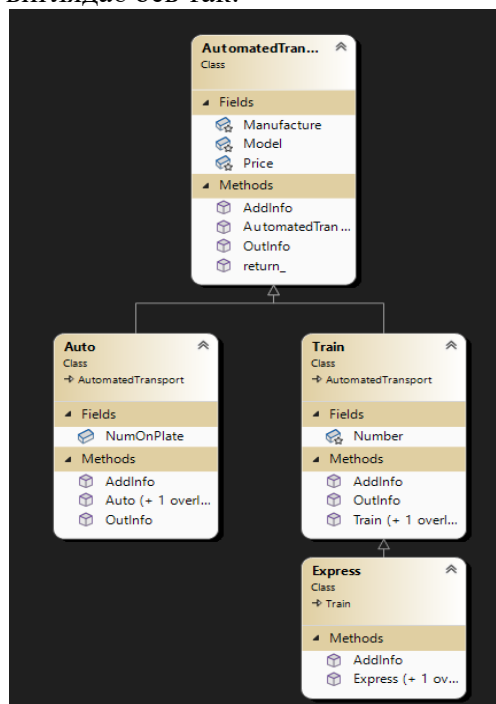
public static string getTyp(AutomatedTransport p)
{
    if (p.GetType() == typeof(Auto)) return "Auto";
    else if (p.GetType() == typeof(Train)) return "Train";
    else if (p.GetType() == typeof(Express)) return "Express";
    else return "Automated Vehicle";
}

```

## 2.8. Пояснення:

2.8.1. Функція Sort() сортує список на основі роботи алгоритму сортування бульбашкою, S() виводить список (використовується для сортування та пошуку), Search() шукає елемент, значення поля якого дорівнює певному елементу, getTyp() повертає тип елемента, .

## 2.9. Діаграма спадковості виглядає ось так:



- Висновок: Ми розробили ієрархію класів згідно отриманого завдання й описали їх алгоритмічною мовою C#, написали фрагмент програми з використанням об'єктів створених класових типів.

|     |      |          |        |      |
|-----|------|----------|--------|------|
|     |      |          |        |      |
| Вим | Арк. | № докум. | Підпис | Дата |